Las personas son un factor clave en los procesos, también necesitamos herramientas que nos ayuden
Proceso definido: asume que podemos repetir el proceso indefinidamente y tener los mismos resultados, usan la predictibilidad para la administración y control. Se adapta el proceso a cada situación particular. Intentan ser procesos completos, que describen lo más posible lo que hay que hacer. La experiencia puede servir para otros proyectos distintos. Creen que se puede detallar de antemano que hay que hacer para realizar el producto.

Procesos empíricos: cubren algunos aspectos, cuando se repite un proceso puede llegar a obtener resultados diferentes. La administración y control es a través de inspecciones. Trabajan en ciclos cortos y usan la retroalimentación. La experiencia es aplicable al mismo equipo, usa la experiencia que va ganando en el proyecto, no la de otros proyectos. El comportamiento de un equipo, para un proyecto, para un producto no es extrapolable.

Ciclo de vida vs proceso: para un proyecto se elige un ciclo de vida para usar el proceso
El ciclo de vida del proyecto te dice cómo encarar las actividades del proceso, cuanto hacer, cada cuanto, ejemplo: cascada, iterativo, espiral (para proyectos)
Hay ciclo de vida de productos (más largos, inician con la idea de crear sw y terminan cuando el producto se retira del mercado ejemplo word) y proyectos (requerimientos, análisis, diseño, etc). En un ciclo de vida de un producto puede haber n ciclo de proyectos (cuando lo mejoro al producto es un nuevo proyecto), cada proyecto genera una versión.

Tipos de ciclos de vida:

- El cascada es secuencial.
- el pud es iterativo/incremental.
- el espiral es el recursivo-> das vueltas pero no generas nuevas versiones del producto, se pone en producción al final recién, solo mostras prototipos.

En procesos definidos puedo elegir cualquier tipo de ciclos de vida y combinarse, procesos empíricos usan el iterativo.

Proceso: conjunto de actividades (estructuradas/guiadas por un objetivo) que toman como entrada un requerimiento y obtienen como salida un producto o servicio. Definición teórica de lo que hay que hacer. El proceso es el nivel más abstracto, define qué se debería hacer. Ese proceso es el que tengo que adaptar en caso de un proceso definido o terminar de armar en caso de uno empírico con la gente involucrada. El proceso se define en términos de roles, para cada proyecto las personas asumen uno o más roles. El proceso es teórico y yo lo instancio en el proyecto para tener un lineamiento de cada persona que rol tiene.

Proyecto: el medio por el cual obtengo un producto de sw, es la unidad organizativa/de gestión,administro los recursos y personas que van a estar trabajando. Para poder existir necesita personas, recursos y una indicación de cómo hacer y que hay que hacer para alcanzar el producto. El proceso da esta definición teórica, el proyecto toma de ahí lo que le hace falta y arma el conjunto de tareas que son necesarias hacer, incorpora al personal que asumen roles definidos en el proceso y arma el alcance del proyecto (trabajo a hacer para cumplir el objetivo).

Ejemplo: el proceso dice que necesito un arquitecto y en el proyecto asigno a x como arquitecto. Una persona puede asumir más de un rol por proyecto y muchas personas pueden asumir el mismo rol.
El proceso dice que tomo requerimientos y el proyecto que x a las 10 am toma requerimientos.
El producto es el software (conocimiento empaquetado, ejemplo: código, una bd, el diseño de la bd, un caso de uso, manuales de usuario-> todo lo que sale como resultado de un proyecto).

- Cada tarea tiene un entregable para saber si se hizo o no, cada artefacto que sale es sw y eso se llama ítem de configuración (según la gestión de configuración)

El ítem de configuración tiene importancia para el equipo y por eso se debe mantener.

Scrum no es un proceso, es un framework de trabajo.

SCM (software configuration management): se aplica en todo el proyecto, busca mantener la integridad del software. Es una disciplina de soporte. Cada cosa a controlar es el ítem de configuración.
Mantener la integridad es responsabilidad de todo el equipo.
Integridad es que satisfaga las necesidades del usuario, rastreable en su ciclo de vida, cumplir con la expectativa de costos y satisfacer criterios de performance.

Esta disciplina asegura la calidad del sw, ayuda a que el resto de disciplinas a que puedan hacer su trabajo, es como un contenedor.

- Repositorio: donde se mantienen los ítems, es el contenedor de ítems. Debería tener una estructura, permite compartir.
- Línea base
- Ramas
- Configuración del sw

software: conocimiento representado en distintos niveles de abstracción y su documentación. Para hacer software usamos un proceso que tiene un conjunto de etapas que generan salidas (artefactos) y son sw.

Clase 3:
El PUD/ RUP (proceso unificado de desarrollo) está formado por la etapa de requerimientos, análisis, diseño, implementación, prueba, despliegue -> cosas para hacer sw.
El sw es todo lo que se genera de las actividades que yo hago,

Disciplinas de gestión vinculadas al proyecto, las técnicas están vinculadas al producto. Y las de soporte (que incluye la gestión de configuración de sw, métricas (que se dividen en el dominio de procesos, proyectos o productos)
El diseño de un prototipo es también sw.

Los cambios de un sw normalmente vienen del ambiente, también están nuestros errores o cosas que dejamos pendientes (desde el inicio podrían hacerse mejor, deudas técnicas)

Gestión de configuración; disciplina que intenta administrar los ítems de configuración a lo largo de su vida. Sirve para que el producto de sw tenga integridad. Para una configuración en particular, necesito;

Involucra para la configuración:

- Identificarla en un momento dado
- Controlar sistemáticamente sus cambios
- Mantener su integridad y origen

identificarla en un momento dado es la versión de todos los items.
Trazabilidad/ rastreabilidad: poder establecer vínculos de información y relacionamiento entre todo lo que conforma aquello que quiero seguir.

El primer artefacto que generamos es la ERS, hay una rastreabilidad pre ers (se usan para responder preguntas como quién te dijo que cambies algo, porque se hizo esto, etc (para saber el origen de algo) y post ers (todo lo que se genera a partir de la ers, modelo de análisis, de diseño, de arquitectura, de bd, codigo, casos de prueba, etc, arma vínculos para saber donde se analizo, donde se diseñó y como se probó)

La gestión de configuración nos permite volver a versiones anteriores. Vuelvo a la última línea base aceptada.

Para que el producto tenga integridad debe satisfacer requerimientos funcionales y no funcionales y ser rastreables.
Diferencia entre ítem y sw: el ítem si o si tiene que ser algo digital, el sw puede estar en papel, si algo está en papel le saco una foto y la guardo en el repositorio y transforme sw en un ítem.

Las herramientas de versionado hacen automáticamente el versionado. No tengo que ponerle por ejemplo en git versión 1.0, lo hace solo, si tengo que mostrárselo a alguien que no tiene acceso al repositorio ahi si le pongo en el nombre la versión.
La variante es un ítem de configuración aparte, y se le pone el mismo nombre de versión que la original solo que se especifica por ejemplo, variante de LINUX.

Yo me bajo la versión a mi espacio local y cuando termino le hago check in/commit.
El plan de gestión de configuración es el primer ítem que tengo que crear, y lo que tiene primero es la estructura, también tienen que estar los nombres de los involucrados y la conformación del control de cambios (su rol y su nombre), cuando se define que hay una nueva línea base.
El cuadro con todas las convenciones del repo va también en el plan de gestión de config

No es lo mismo un ítem que una línea base, ni tampoco es lo mismo una gestión de configuración que una línea base.

Línea base: item o conjunto de ítems que fueron validados y probados y conformados como punto de referencia para el resto del equipo. No necesariamente está formada por todos los ítems del repositorio ni por todas sus versiones Todo el mundo que está afectado sobre el cambio de la línea base se le va a informar.

A lo largo del desarrollo del producto se puede tener más de una línea base.
La línea base también es un ítem de configuración. El control de cambios incluye a todos los involucrados con los cambios, necesitamos representantes del mismo equipo (analista funcional representante de los requerimientos, líder de la arquitectura-> líder técnico; el que hace de gestor de configuración, cliente (depende de la característica del cambio, si lo originó el cliente está bueno que este y para que sepa el impacto que va a tener el cambio). Nadie toca la línea base si alguien no autorizó el cambio.

El comité tiene que autorizar el cambio y ver si lo que se hizo fue lo que se autorizó.
No mezclar items de producto y de proyecto. El proyecto está formado por iteraciones (ciclo de vida en cascada tengo 1 iteración, incremental tengo más de 1). Cada iteración tiene un plan de iteración que su ciclo de vida es más corto que el del proyecto.

Ejemplo de un documento de producto ERS_clima.doc, el nombre del proyecto no sea el mismo que el del producto-> PP_matrix.doc (uno solo por proyecto), plan de iteración-> pi_matrix<<nro>>doc

Una rama puede ser integrada o descartada.
Una variante de un ítem es un cambio que no requiere una nueva versión-> por ejemplo la versión 1,5 y 1,6 son dos variantes.

Caso de uso para una iteración es un ítem de producto e iteración. Si el ítem muere con la iteración entonces es de iteración, si muere con el proyecto es ítem de proyecto y si lo tengo que mantener mientras viva el producto es de producto.
Si el ítem se puede repetir se usan las etiquetas variables con <<>>

PRIMER TEMA DEL PARCIAL PRÁCTICO: USER STORIES

El agilismo adhiere a procesos empíricos
Pilares del empirismo: inspección, adaptación (necesidad de cambios), transparencia-> equipos empoderados y autogestionados.

Infografía: más imagen que texto
Valoramos más lo que está arriba que lo que está abajo.



Valoramos más individuos e interacciones que procesos y herramientas. Para nosotros es más importante la gente y los vínculos que se establecen entre ellos, el equipo debe poder resolver sus conflictos y autogestionarse.
Todas nuestras acciones deben apuntar a generar valor por ende, el software funcionando es más valorado que la documentación extensiva.
Valoramos más colaborar con el cliente (es parte del equipo, se trata del PO) por sobre negociaciones contractuales.
Scrum funciona a nivel de proyecto pero nunca dice que no haya que documentar

Principios más importantes:

- técnicos (diseñadores y desarrolladores entre otros) y no técnicos (responsables de negocios, seria en scrum el PO, maneja el presupuesto por ende es el que decide lo que se hace, decide el que no el cómo) trabajando juntos. La responsabilidad del producto la tiene el po y la responsabilidad de cómo se construye la tiene el equipo. Este principio es la clave del éxito.
- el mejor medio es la comunicación cara a cara
- Las mejores arquitecturas requisitos y diseños **emergen** de equipos autoorganizados. Las decisiones las toma el equipo y no un superior.

El agilismo es un pensamiento que define buenas prácticas para trabajar
Cuando hablamos de **requerimientos ágiles** nos focalizamos en el valor, resultados, porque tenemos que aceptar la realidad y empezar a fabricar el producto para ganar experiencia y obtener la retroalimentación. En cada iteración voy a trabajar ciertos requerimientos y solo de esos necesito el máximo detalle.
Principios ágiles que siguen los requerimientos ágiles: aceptar que los requisitos cambian, nuestra mayor prioridad es satisfacer al cliente, comunicación cara a cara (uno de los componentes fundamentales de la US es la conversación), etc.

El agilismo dice que hay que ser expertos en el negocio, debemos entenderlo mucho para generar un outcome. Y no necesitas tener el 100% del producto especificado para empezar a hacer el producto, no solo no hace falta sino que no es posible porque el 50% de los requerimientos emergen conforme el producto se esté construyendo.



En PUD se definen los casos de uso que van en la iteración, y no se termina hasta no completarlos todos, en cambio, en scrum dejo fijo el tiempo y el alcance que voy a entregar tiene que ver con los requerimientos más arriba en el PB que voy a llegar a cumplir con el tiempo y recursos disponibles.

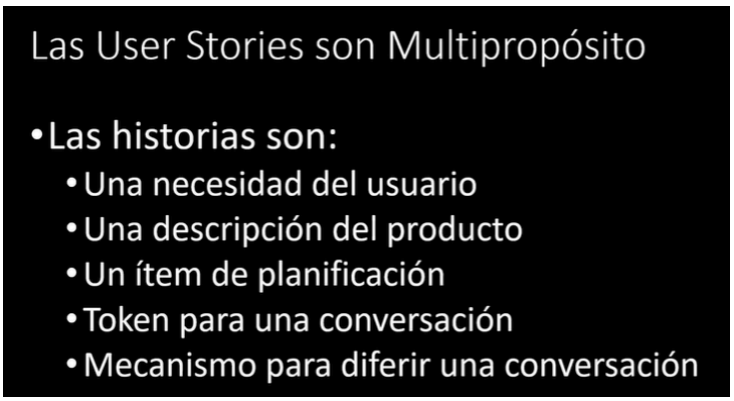
Artefacto principal del PO: product backlog, cada cosa del pb son ítem de product backlog. Cada uno elige la técnica que se usa para trabajar los ítem de pb (user stories, épicas, defectos, deudas técnicas). Las ideas menos formadas están más abajo.
El enfoque ágil es compatible con el lean por ende el agile toma lo de just in time que dice que no necesitamos conocer al principio todo el producto ni tenerlo especificado desde el principio
El product backlog nunca va a estar completo porque una vez que se implementó algo no vuelve a la lista, no es como una ERS. Se hacen las funcionalidades prioritarias primero, se habla de prioridad no de importancia porque todas son importantes.

Con los requerimientos ágiles apuntamos a atender: cuando trabajamos en ágil no profundizamos en los requerimientos del dominio de la solución. Usamos los requerimientos del negocio para a partir de ahí construir una solución que pueda entregarsela al cliente con entregas tempranas.



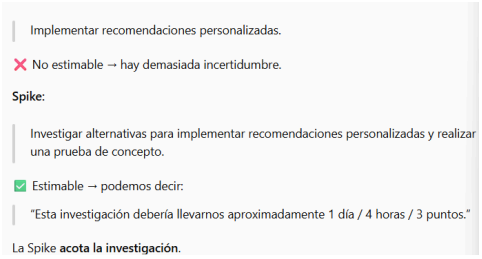
User story: descripción corta de una funcionalidad que necesita el usuario, conformada por las 3 c: card, conversation (la más importante) y la confirmation (pruebas de aceptación, lo que tenes que identificar-> cómo lo vas a probar para demostrar que cumple con lo que el po necesita).
Está mal en la historia de usuario poner como usuario, necesito poner el rol de esa persona en el negocio.
La card es la parte visible de la historia de usuario, los criterios de aceptación tienen que ver con la confirmación.
Atrás van identificados los escenarios que el PO va a probar para confirmar si acepta o no el avance.
Una user es una funcionalidad que debe hacer algo de valor para el po por ende tiene que cortarse verticalmente, no puedo entregar la bd.
Un PO por producto, si no puede estar presente el po real se designa un representante que tenga conocimiento del negocio y pueda tomar decisiones, siempre tiene que venir del negocio.
Las pruebas de usuario no necesito que las escriba el usuario pero sí que las piense el usuario.

Las tech stories (los requerimientos no funcionales transversales, los que no son transversales se ponen como criterio de aceptación) no se entregan solas, ejemplo la performance o que sea mobile. Entonces cuando la voy a estimar voy a tener en cuenta todos los RNF que voy a necesitar.
Una US COMPLETA tiene la expresión, los criterios de aceptación, las pruebas (algunas que fallen y otras pasen) y la frase verbal



Cuando la user esté en lo alto del PB se genera como un “token” para una conversación porque implica sentarse a hablar a refinar los requerimientos y también nos permite diferirla (ejemplo estoy armando el gps y me acuerdo de otra funcionalidad que más tarde definiremos mejor). Y sirve como ítem de planificación porque cuando vaya a implementar debo usarla para planificar.
La alta, baja y modificación son 3 user distintas.
Tema: organizador de user, ejemplo tema clientes y ahi adentro van todas las users de los clientes

La definition of ready se define a nivel equipo y para ello se ayudan con invest.
Invest es para controlar lo mínimo que una user debe cumplir para estar en condiciones de desarrollarse.
SPIKE es una user que no se puede estimar, porque tiene problemas funcionales o de negocio que no están claras, o porque tiene problemas técnicos (combinación de tecnologías o cosas que no hiciste nunca o no se probaron nunca). Se ponen en el product backlog como spike y alguien debe investigarla y alguien resolverla (se puede hacer con un prototipo). La spike en sí debe ser estimable y debo poder demostrar que resolvi el problema



El agilismo trabaja con gestión binaria asique o la user está hecha o nada, no gestiona porcentajes de avance para eliminar el “falta poco”.

Poner la user como usuario cuando son user de logueo por ejemplo pero sino no, ayuda a definir los roles y los permisos de cada rol

Una user story no es una especificación del requerimiento, para eso está el CU
No poner quiero poder como user, poner quiero o puedo

Los requerimientos son ágiles porque varían a lo largo de todo un proyecto.

Clase 1/9- estimaciones ágiles:

Estimación: se está trabajando sobre una incertidumbre, nos falta información (de la gente que va a trabajar, su nivel de expertis, los requerimientos, determinar efectivamente cual es el producto que el cliente necesita).
No es un compromiso, se confunde lo que quiero lograr con la estimación. Se tiene un factor de probabilidad, cualquier estimación debe ir acompañada de su probabilidad de cumplimiento.

Desperdicio: algo que no genera valor.
Ninguna decisión se puede tomar en un ambiente aislado del contexto-> no hay que ser dogmático, en función del contexto hacer las cosas.
El objetivo del negocio está formulado por alguien, por eso es subjetivo
Estimación es una predicción analítica o imparcial del esfuerzo o costo. La objetividad viene porque trato de usar parámetros lo más sistemáticos posibles para llegar a un número pero no se asocia a la certeza.

Compromiso: es lo que esperamos lograr con la planificación, la planificación toma como entrada una estimación, y la planificación incluye más aspectos que la estimación como: lineamientos del negocio. Cuanto más diferencia haya entre lo estimado y lo planificado, mayor riesgo. La planificación implica un compromiso, la estimación no.

Normalmente se parte de haber asociado estimación con compromiso, cronograma irreal dictado por el negocio. Cómo partiste de un cronograma subestimado (por ejemplo que el negocio te dijo que sea en menos tiempo, te fuerzan a hacer un cronograma ajustado a las necesidades del negocio), como consecuencia se recortan actividades tempranas por estar apurado-> viene el: lo hagamos asi nomas total después lo mejoramos pero nunca hay tiempo para hacer eso entonces queda el código sucio y rápido y queda con defectos que se detectan en el testing o los detecta el cliente.

Se estima antes de arrancar un proyecto, sería como la pre estimación y después se estima en cada sprint para ver que us incorporar.

El cono de la incertidumbre: dice que las estimaciones se piden demasiado pronto porque el cliente las pide, se dan al principio para armar la propuesta al cliente y si después el cliente no acepta la propuesta se pierde la plata y el esfuerzo invertido, acá la incertidumbre es mayor. Agilismo propone que la certeza venga con la retroalimentación del software funcionando, entonces a más iteraciones - más certeza.
muy poco proyectos terminan consumiendo menos de lo que estimaron, y normalmente es porque arrancaron sobreestimados (decir que necesito más tiempo del que realmente necesito)

Buena estimación:

- expresada en rango, nunca decir voy a terminar el 8 de septiembre. Los rangos de oscilación tienen que ser razonables, un 20% para arriba y para abajo. (ejemplo entre 800 y 1200 horas)
- incluye un porcentaje de probabilidad
- La estimación no debe estar modificada para encajar en una fecha deseada.

Porque son ágiles las estimaciones ágiles:
Porque estima el equipo de desarrollo entero y no una sola persona. Porque estimas a lo largo de todo el desarrollo iteración por iteración (en tradicional estimas 1 vez al principio, pueden existir revisiones y reestimaciones, pero la planificación y estimación inicial tienen mucho más peso y se intenta definir gran

parte del proyecto anticipadamente). A medida que el equipo desarrolla y obtiene más información, puede cambiar sus estimaciones. También es porque estiman de forma relativa comparando US entre sí (tamaño, esfuerzo o complejidad), y no estiman poniéndoles un tiempo como en el modelo tradicional

Trabajan con estimaciones relativas: somos mejores estimando por comparación, el agilismo dice que no hagamos una estimación absoluta (cuantos metros de alto tiene el edificio, sin compararlo con otro).
La complejidad del software crece exponencialmente por eso usamos la escala de fibonacci (sumamos los dos anteriores para obtener el número)
La unidad de medidas para estimar las US son los story points (SP). Las us se estiman en sp. Nosotros estimamos el tamaño de la user, porque cuando estara o el precio puede derivar del tamaño.

Tamaño refiere al producto a construir y el esfuerzo se mide con horas, personas lineales (1 sola persona haciendo una sola cosa a la vez, no considera solapamientos). Esa estimación de esfuerzo se usa para estimar el tiempo (para responder el cuando va a estar). Primero se define el que, después el cómo y de ahí deriva el cuanto sale y cuanto tiempo lleva (en tradicional)
En ágil se dice que el que debe estimar es el que hace el trabajo (en tradicional estima el líder del proyecto y tiene la responsabilidad) y por eso son los equipos autogestionados.

Escala de estimación para estimar los ítems en el product backlog: talle de remera. Las dimensiones se usan para justificar porque sería un M o L.

Un punto de historia tiene 3 dimensiones que pueden presentarse en distintas proporciones (dimensiones) dependiendo de la user: complejidad, esfuerzo e incertidumbre.

La métrica de ágil es velocidad: cantidad de puntos de historia que el PO aceptó al final de la iteración (el acepta las us y eso suma sp) y esta no se estima , se calcula sumando y multiplicando al momento de presentar el incremento, osea al final de cada sprint.
Si tengo us1= 3 sp, us2= 5 sp, us3= 1sp y solo la us1 se aceptó entonces sumo 3, el resto vuelve al producto backlog.

Pero todavía no sabemos nuestra velocidad.

Hacemos el Sprint 1:

> Completamos 20 puntos.

Sprint 2:

> Completamos 18 puntos.

Sprint 3:

> Completamos 22 puntos.

Sprint 4:

> Completamos 20 puntos.

Nuestra velocidad está alrededor de:

20 puntos/Sprint

Ahora sí podemos hacer una predicción:

100 puntos ÷ 20 puntos/Sprint = 5 Sprints

Si cada Sprint dura 2 semanas:

5 × 2 = 10 semanas aproximadamente.

Antes de tener velocidad real, podés hacer una previsión inicial usando una velocidad suposicional o esperada, pero va a ser menos confiable. Por eso Agile distingue:
Antes de empezar: "Creemos que aproximadamente serán 5 Sprints.
"Después de tener datos:"Según nuestra velocidad real, probablemente sean 5,2 Sprints."

Poker estimation: método que utiliza el equipo para asignar los Story Points a las User Stories.
El equipo elige una user canónica (se usa como referencia para comparar las otras user) y se le asocia un punto de historia. El ½ es para los equipos que usan 1 como canónica (la story más sencilla). Se estima en privado con el equipo para no sesgar ni verse sesgado por el resto. (el número que use para estimar debe estar en la escala de fibonacci, normalmente se usa fibonacci en las cartas pero puede ser que no)
Cuando se llegó a un consenso se empieza el mismo proceso para la siguiente user, se corta cuando tengo la cantidad de historias suficientes para el sprint.
Una user con 13 puntos normalmente no entraría en un sprint, hay que partirla.

Los Story Points son una medida relativa del tamaño de una User Story. No dicen: "Esta historia tarda 8 horas." Sino: "Esta historia es bastante más grande que una historia de 3 puntos." La secuencia de fibonacci se usa como escala para la story point, al igual que el tamaño de camisa. Son alternativas para expresar el tamaño relativo.

Reestimamos cuando como equipo notamos que se cometió un error, se re estima si cambio el tamaño de la user. Si llego al final del sprint, la user vuelve a ingresar al product backlog con el mismo tamaño que tenía.

Se negocian los compromisos no las estimaciones.
La planificación del release debe responder cuantos sprints necesitas para entregar el mvp.
En scrum la duración del sprint es fija (máximo 1 mes), no se mueve. El agilismo dice que la duración es fija y el alcance es variable, si no terminaste todo entrega hasta donde tengas.

MVP: producto mínimo viable (factibilidad), para validar una hipótesis, alguien va a querer el producto, sirve para alguien?. El índice de retención va a determinar si es viable para determinar si vale la pena hacer el producto. Puede ser un prototipo, un video, una maqueta, solo queres mostrarle a los clientes/ interesados para ver si el producto va a ser exitoso o no. El objetivo no es comercializarlo, solo se enfoca en el aprendizaje. Se define al comienzo del proyecto, cuando estás definiendo qué producto querés construir y cuál es el alcance inicial.

MMP: de comercializable, esto ya si tiene que ser software funcionando, es la cantidad mínima de características que el producto tiene que hacer para generar una versión que se pueda poner en producción, que tenga valor.
Todo el mundo habla de mvp cuando en realidad se refiere a mmp.
Hay que priorizar un conjunto limitado de funcionalidades.
Se lanza cuando definimos las condiciones para tener un mmp, ejemplo llegar a tal user.

MLP: la L es lovely, tiene que ver con que hay tanta oferta que hay una sola oportunidad para que el usuario disfrute usar el producto, no alcanza que tenga la funcionalidad sino que debe hacerlo feliz. Te lleva a tener en cuenta funcionalidades que antes decíamos que las veíamos después.

Defino aprox hasta que user es mi mvp y una vez que esta lo lanzas.
Puedo usar los 3 juntos o solo algunos.

Claves para desarrollar un mvp:

- que podamos obtener el feedback lo más rápido posibles
- seguir el ciclo construir, adaptar y cambiar-> viene por uno de los pilares del empirismo (sus pilares son: transparencia, adaptación)
- reducir los riesgos: busco con el menor costo, obtener la mayor info

Un mvp me permite visualizar lo que voy a desarrollar en un proyecto de software, y ver que tan grande es lo que voy a construir

En enfoque tradicionales se estima: tamaño (que tan grande va a ser lo que vamos a desarrollar, se mide en funcionalidades/ características), esfuerzo (horas personas lineales, no permite trabajo en paralelo, tiempo en que una persona tarda en hacer una actividad), costo, calendario - tiempo (días, meses)
En tradicional también se va estimando a lo largo de todo el proyecto.
En enfoque ágil se estima: tamaño de las user, la unidad de medida es en storys point.

Story point: es la suma de la complejidad (que tan difícil es el problema que trato de resolver, relacionada con la naturaleza del problema o con la tecnología necesitada), esfuerzo (hora lineales que me lleva desarrollar una user) e incertidumbre (nivel de conocimiento que tiene el equipo, desde el negocio hasta las tecnologías que necesito).
El número que se usa en poker planning puede seguir la escala lineal o de fibonacci o de talles de remera (acá usamos fibonacci) y se toma una referencia y en base a esa se compara el resto, en la cátedra se toma como referencia la más fácil (con menor valor, que la combinación de esfuerzo, incertidumbre y complejidad sea baja).

Cuando estimamos ahí tenemos en cuenta la expertiz del equipo, no subestimar ni sobreestimar.

La clave de ponerle el valor es la justificación del porqué dio eso.
Cómo definir el mvp: debe estar declarada en forma de hipótesis, se usa la palabra alcance (las cosas que cubre y las que no). Lo vamos a definir por medio de un listado de frases verbales de las user stories.
Se define; alcance: incluye (un listado, no incluye (módulo) cada uno con su justificación de porqué se incluye o no. Y si lo vuelvo a leer y me pregunto por qué?, la justificación está incompleta.

Clase 8/9 - componentes de proyecto:

El proyecto es una unidad de trabajo y es el medio por el cual se obtiene un producto.
Empírico: la actitud que tiene frente a los cambios es que rapido quiero retroalimentación para adaptar

Tipos de ciclo de vida: iterativo, secuencial, recursivo. Dentro del tipo de ciclo secuencial conocemos el cascada, dentro del iterativo está el iterativo incremental. Revisar esto

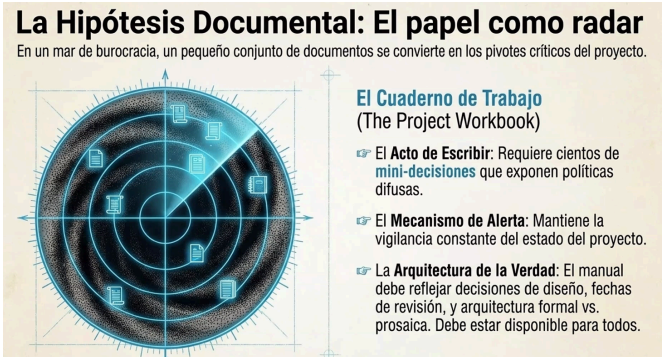
Diferencia entre proceso y ciclo de vida: El proceso define las cosas que hay que hacer (actividades, entradas, salidas, roles). Es la definición teórica. Mientras que el ciclo de vida del proyecto determina cuánto de cada actividad del proceso se hará cada vez, y los puntos de control (te dice como usar el proceso)
El PUD recomienda un ciclo de vida iterativo incremental.
En procesos empíricos, sólo se puede usar el ciclo iterativo porque se basa en generar conocimiento y experiencia rápidamente.

Si el resultado no es único, no es un proyecto. Además, debe tener una fecha de inicio y de fin.
Los proyectos deben tener un objetivo claro (todos entendemos lo mismo) y alcanzable (haciendo lo mejor que podemos todos, el objetivo puede lograrse). NO puede haber un objetivo alcanzable pero que no sea claro.

UN EJEMPLO ES ALGO CONCRETO, no hacerlos generales.

Lo importante de la planificación no es el plan de proyecto (artefacto), lo valioso es lo que aprendes mientras planificas.

En el paper de no silver bullet lo accidental apunta más a ayudar en el como y no en el que (el real problema)
Si agregamos personas a un proyecto atrasado, es peor porque gastamos tiempo en capacitarlos.
Hay una deformación personal al “y ya que estamos” entonces construimos desperdicio y perdemos el enfoque principal y sacrificamos velocidad en funciones rara vez usada.
Mientras más gente, más ruido de comunicación, más interpretaciones lo cual lleva a problemas.
LEER LA GUÍA DE SCRUM 2020 Y 2026, porque scrum da un minimo y maximo de gente?



algo de información documentada tiene que haber, el cómo no importa (sin en un pizarrón, foto o lo que sea), importa el que. La escritura requiere un nivel de precisión distinto que el habla.

Un producto de software tiene calidad si satisface las necesidades y expectativas del cliente

Disciplinas que integran la administración de proyectos: planificar y monitorear, se puede planificar y no monitorear pero no se puede monitorear sin planificar, porque el control es una actividad de comparación y necesito saber que tengo que comparar y un estándar contra qué compararlo.
Entonces, la administración se encarga de la planificación y monitoreo de un proyecto.
Las dimensiones sobre las que se mueve un proyecto son alcances (requerimientos), tiempo y costo (que son los recursos). La disciplina de gestión de proyectos busca un equilibrio entre las dimensiones, sin afectar la calidad. El cliente puede controlar el costo y el alcance, pero la única variable que debería controlar es la de que quiere (alcance)
En la gestión tradicional se derivan los recursos y el tiempo en función de los alcances. Y como el cliente puede venir y pedir algo más sin querer acceder a que se requiere más tiempo y recursos, se empieza a sacrificar la calidad.
No siempre que cambie algo voy a modificar las 3 dimensiones, puedo querer agregar algo y sacar otra cosa.
El líder de proyecto coordina y tiene a cargo el equipo del proyecto, habla con el cliente, con los jefes, etc (es la interfaz entre el equipo y los que tienen algo para decir).
En ágil se plantea que las personas sean multidisciplinarias, ejemplo que sean fullstack, que puedan negociar con el cliente, la documentación, etc. Dejar la user para cumplir con el criterio de ya terminado

Necesitamos un plan para que nos guíe el camino que vamos a emprender.
El alcance del proyecto y del producto, no son lo mismo. El alcance del proyecto se escribe en el plan del proyecto y el alcance del producto se especifica en la ERS. Se condicionan pero no son lo mismo.
ESTIMACIONES DE SOFTWARE

La gestión tradicional tiene más foco en la precisión de la estimación, apunta a estimaciones absolutas. Debe seguir el siguiente orden

- Tamaño: del producto, apunta a saber qué se debe hacer, independiente de quién lo hace y la plata del cliente. En CU, requerimientos
- Esfuerzo: en base al tamaño determino las horas persona lineales (1 sola persona trabaja haciendo una cosa a la vez)
- Calendario: transformo las horas en una estimación de cuándo, tengo en cuenta cantidad de personas, trabajo en paralelo, jornada de trabajo, disponibilidad del equipo. Derivo el calendario
- Costo: lo último a estimar porque se calcula en base a las anteriores estimaciones
- Recursos Críticos: no siempre están. Se calcula que se va a necesitar más de lo que se dispone

Se deriva el calendario a partir del esfuerzo
Riesgo tiene un factor de probabilidad y se mide por la exposición (que tiene en cuenta la probabilidad y el impacto, prob*impacto). El riesgo se estima porque tiene una probabilidad. Riesgo no es problema, el hecho de que no alcance el tiempo es un problema, o empezar un proyecto que no se tiene en claro (no tiene probabilidad, hay certeza, es un problema). En un proyecto sano se espera una gestión proactiva de los riesgos (que también genera conocimiento). Algunos riesgos es que se vaya gente clave del proyecto, o tomar mal los requerimientos (el mas jodido), la tecnología
El cálculo de exposición sirve para comparar un riesgo con otro.

Clase 15/9 scrum

scrum es un marco de trabajo basado en el empirismo, genera valor a través de eventos iterativos para la entrega de valor. Resuelve la creación de productos complejos de manera adaptativa (no tenemos claro qué producto crearemos). Es una manera de organizarnos para crear algo, es un framework de trabajo genérico, nace para el sw pero se extrapola para otras disciplinas, da pautas para como organizar el trabajo. NO ES UNA METODOLOGÍA NI UN PROCESO COMPLETO COMO EL PUD.

Lo que dice ahí hay que hacerlo como dice ahí, sino no puedo decir que hago scrum.

Cubito celeste es el producto, cuadrados verdes son tareas.

Equipo scrum : po, developers y scrum master. Hay un representante del negocio y técnicos (principio de técnicos y no técnicos trabajando juntos). Y más afuera están los supporters y más afuera los stakeholders

Versión 2026:

Se cambia desarrollador por: Product developer para no asociarlo solo con código.

Aparece un rol nuevo: supporter- vínculo que aparece porque un equipo ágil no sobrevive en una organización que no es ágil. No podes sostener una cultura (agilismo) si el contexto no tiene esa alineación.

Profesionalismo: tener en cuenta la ética de trabajo, responsabilidad (en realidad es accountability, es hacerse cargo del compromiso hasta que se cumpla), capacidad de autogestionarse, transparencia, comprometerse.

Define precisamente que es un stakeholder.

Equipos de 7+-2. Hasta 9 personas y no menos de 5.

Hace la diferencia entre output y outcome: output es lo que sale de la iteración, el sw. El outcome es el valor del negocio que el output género. Si no genero valor no sirve. Divide el definition de done entre el output y el outcome.

Compromiso técnico es el output, compromiso de valor es outcome y manda sobre todo el PO.

Scrum funciona en el cuadrado del proyecto

El framework pide que la gente tenga trabajo dedicado (un solo proyecto a la vez), no se puede hacer foco en más de 1 proyecto a la vez.

El product developer es el único que puede ser desarrollado con ia o con su ayuda, hay un compromiso que dice que la ia no trabaja sola (human in the loop, es parte del accountability, parte de la responsabilidad ya que siempre tiene que haber supervisión humana)

Sprint es un periodo de tiempo FIJO, es la ceremonia contenedora de las otras y funciona como time box, establece duraciones máximas de las actividades innegociables, se haya cumplido el objetivo o no se termina (máximo 1 mes y minimo 1 semana porque sino no alcanzó a generar el valor).

Sprint planning: definimos el sprint goal, que vamos a hacer, se hace la estimación. Como entrada necesitamos los ítems del product backlog, referencias de velocidades de sprints anteriores si es que las hay, capacidad, objetivos del sprint. Genera el sprint goal y sprint backlog, termina el sprint y se borra, define el compromiso de que incremento de producto voy a generar para cumplir con el objetivo del sprint.

Por sprint voy a tener un incremento del producto que pueden ser muchas funcionalidades, es como una nueva versión.
Para ver cuantos ítems paso del PB al sprint tengo en cuenta la capacidad del equipo, a cuánto trabajo puede comprometerse el equipo por sprint. La capacidad se puede medir en horas ideales o storys points (los equipos más maduros trabajan con puntos de historia), la versión 6 dice que si tenes una capacidad de 500 horas conviene comprometerse a un poco menos.

Métricas más importantes:

- velocidad porque mide producto, la mejor métrica es la que muestra software funcionando. No se estima, siempre se mide únicamente con puntos de historias
- La otra es la capacidad que se estima al principio del sprint para ver a que me voy a comprometer como equipo. Se puede medir con puntos de historia o horas ideales.

Después del planning va la daily: reuniones de no más de 15 minutos, sirve para inspeccionar y adaptar (pilares del empirismo). Es una reunión de sincronización y es importante para saber si estamos encaminados al objetivo del sprint, se responde al qué hiciste, qué pensas hacer hoy (asignaciones diferidas, el desarrollador se compromete)
No se negocia calidad ni producto terminado (o cumplo definition of done o no, si no lo cumple no las presento a esas user)
En la review inspeccionamos y adaptamos con foco al producto. La última ceremonia (retrospective) tiene foco en el proceso, como se hicieron las cosas. Todas las ceremonias se hace ADENTRO del sprint.
Durante el sprint no hay modificaciones de los requerimientos (salvo que sean mínimos), se hacen después.

Stakeholder: todas las personas que tienen algún interés con el producto (entre ellos los sponsors, gerente de producto, inversores, el gobierno)

Refinamiento del product backlog, otra ceremonia: en las primeras sesiones puede estar solo el PO, directivos, expertos del dominio, se trabaja primero en la visión del producto y se hace una configuración inicial del backlog.
El invest se usa para definir la calidad de la user y ver si se la puede incluir en el sprint backlog, se reserva un porcentaje del sprint para hacer refinamiento, es una actividad continua (la convoco cada vez que quiero saber algo del producto). Se hace al principio pero también por ejemplo si surge una duda en la daily post daily hago una sesión de refinamiento.

El único rol que no se puede mezclar en la misma persona es el PO con los otros, sobre todo con el scrum master, uno es un rol de negocio y el otro es técnico.

2. Proyecto Fallido: La plataforma Healthcare.gov (Estados Unidos)

- **Qué era:** Un portal web centralizado lanzado por el gobierno de EE.UU. para que los ciudadanos pudieran cotizar y comprar seguros médicos obligatorios de forma online.
- **Por qué fracasó:** El día de su lanzamiento colapsó masivamente. Millones de usuarios intentaron entrar y la página se congelaba, daba errores de servidor y era totalmente inutilizable.
- **Conexión con la crisis del software:** Muestra crudamente los **factores actuales de la crisis (demandas crecientes y fallas en la toma de requerimientos)**. El proyecto intentó gestionarse con métodos tradicionales rígidos (con plazos fijos y alcance inamovible) sin validar iterativamente la concurrencia ni las verdaderas necesidades de carga de los usuarios. Al no construir con un enfoque flexible o validaciones tempranas, el producto final no sirvió para el propósito del negocio.

- **Fallos en la toma de requerimientos:** El resumen de la clase señala explícitamente que **el 80% de los fracasos de los proyectos se deben a fallas en la toma de requerimientos** y a planificar basándose en datos insuficientes o supuestos sin consultar datos históricos. En Healthcare.gov, no se relevaron ni especificaron correctamente las necesidades reales de integración, arquitectura y seguridad desde el principio entre las múltiples agencias gubernamentales y contratistas privados involucrados. PDF
- **Falta de pruebas y validación temprana:** Al intentar aplicar un enfoque tradicional rígido (en lugar de entregas incrementales o pruebas tempranas con usuarios), el sistema nunca se probó adecuadamente bajo condiciones de estrés masivo antes de salir a producción. PDF

Por lo tanto, al no hacer un buen relevamiento y no poner a prueba el producto de forma iterativa, el sistema colapsó masivamente el primer día al enfrentarse al uso masivo real, reflejando exactamente las causas de fracaso que estudia la ingeniería de software. PDF

3. Proyecto Exitoso: El desarrollo de Linux / Git (Software Libre y Colaborativo)

- **Qué era:** Un sistema operativo y un sistema de control de versiones descentralizado creados para gestionar el código fuente de manera colaborativa a nivel mundial.
- **Por qué triunfó:** Permitió que miles de desarrolladores distribuidos en todo el planeta escribieran, probaran e integraran código de forma continua, manteniendo la estabilidad y la trazabilidad absoluta de cada modificación.
- **Conexión con la crisis del software:** Representa la solución a la crisis mediante la aplicación de **disciplinas de soporte y gestión de configuración modernas (SCM)**. Demuestra que para dominar la mutabilidad y la complejidad del software no sirve un plan estático de una sola persona, sino repositorios descentralizados, integración continua, control estricto de versiones y una estructura transparente donde todo el equipo mantiene la integridad del producto.

Para conectar el triunfo de **Git** directamente con la **crisis del software**, debemos recordar que uno de los mayores problemas de dicha crisis era la **falta de disciplina, el caos en el control de los cambios y la dificultad para manejar la mutabilidad y la complejidad** del software. PDF

Git se convirtió en una herramienta clave para superar la crisis del software por las siguientes razones:

- **Controla la mutabilidad y evita la pérdida de cambios:** Como el software está sujeto a constantes cambios a lo largo del tiempo, Git soluciona el problema histórico de la superposición de modificaciones, el doble mantenimiento o la pérdida de componentes mediante un registro histórico exacto de cada versión. PDF
- **Reemplaza la burocracia por un control descentralizado y seguro:** En la crisis del software, los intentos de control eran tan rígidos o manuales que terminaban siendo burocráticos y lentos. Git descentraliza el repositorio dándole una copia completa a cada desarrollador, permitiendo trabajar en paralelo mediante ramas (*branches*) y fusionarlas (*merge*) de forma ordenada sin romper la estabilidad del sistema principal. PDF
- **Garantiza la integridad del producto:** La gestión de configuración busca mantener la integridad del software (que el producto haga lo que debe y sea rastreable). Git aporta esa "columna vertebral" técnica que asegura que las líneas base de código sean confiables, permitiendo que los equipos se organicen mejor y reduzcan drásticamente los errores humanos que alimentaban la crisis. PDF + 1

- **Entendió el problema raíz de la crisis:** Linus Torvalds (su creador) identificó que las herramientas de control de versiones existentes eran lentas, costosas o centralizadas, lo cual agravaba el caos y la pérdida de código en proyectos grandes.
- **Aportó disciplina y soporte técnico:** Git se convirtió en un caso de éxito global porque estructuró de manera perfecta la **Gestión de Configuración del Software (SCM)**. Permitió automatizar ramas, fusiones (*merges*) y trazabilidad, dándole a los equipos la estabilidad técnica necesaria para que el desarrollo iterativo e incremental (como el que exige la agilidad) sea realmente posible y seguro. PDF

REVISAR SCRUM 2026

No entiendo bien para que es la sprint review, es para mostrarle al po la users y que puedan salir a producción o no siempre hace falta mostrarles. Siempre hace falta mostrarle al po y la sprint review es

específica para mostrarle al po porque el dod es solo para un control interno del equipo y en la sprint review el po puede llamar más gente del negocio para chequear lo que se hace.

El sprint backlog se puede modificar durante el sprint? si veo que me falta tiempo o me sobra tiempo puedo agregar o quitar cosas sin modificar el sprint goal? Se puede modificar el sprint si es algo mínimo y no modifica el sprint goal, pero se supone que no debería haber modificaciones porque es time box, se supone que no deberías terminar antes por ende no debería agregar users.

¿ va el tema de métricas? qué son, cómo se clasifican o solo van las métricas que vimos de ágil. No va métricas

La user si o si debe aportar valor, asique si divido una épica en user sin valor no sirve.
En un product backlog puedo poner épicas, temas, us, defectos, deuda técnica, tech stories. No hay tareas o actividades (esas van en el sprint backlog)
El spike puede estar en el sprint backlog porque sino nunca nadie se ocuparía de resolverla, pero no meter en el mismo sprint la user que se relaciona con la spike. El spike debe ser estimable porque debo poder darle un espacio un tiempo de trabajo

El refinamiento del producto es para la detección de requerimientos y para que el PO te conteste, sacar las dudas y relevar lo que hay que hacer, mantener el pb organizado y actualizado. Se debe reservar el 10% del sprint para el refinamiento. No encaja en ningún momento del ciclo de vida del sprint porque lo necesitas todo el tiempo.

En mvp o mvf hay algo para validar, mientras que en mmp es la mínima versión que tenga valor para el cliente.
El producto termina siendo una sumatoria de mmr, la suma de todos los releases termina armando un roadmap. Normalmente el primer release es el mvp pero si estamos en producción es mmp
Sobre un producto ya en uso se puede crear un mvp.

En 2026 scrum el desarrollador es el product developer, que puede ser una ia pero por el human in the loop siempre debe haber un humano que chequee.

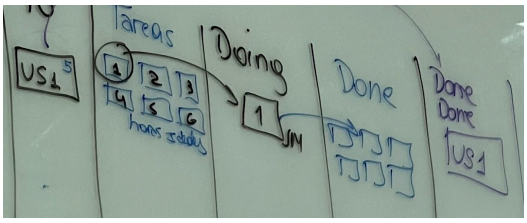
Ciclo de vida de proyecto = ciclo de vida de desarrollo

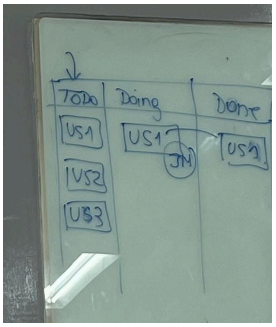
El nivel de planificación más chico es la daily, después el sprint planning (el alcance es la iteración y el artefacto es el sprint backlog), despues esta la planificación del release (tiene que decir que producto vas a entregar en este release, describe las características que entran al release y cuando se va a entregar).
Un plan de release termina siendo una sumatoria de release, derivas la duración del release de la cantidad de sprints. En scrum si se planifica y mucho porque hay que re planificar cada vez que se cambia
Por arriba del plan de release está el plan de producto, dice que este release para tal fecha este otro y cual es el alcance de cada uno.
Del roadmap (es a nivel de producto te dice para esta fecha deberían estar estas características; y nos da el pie para armar el plan de producto) se construye el plan de producto y de ahi para abajo, planifico el primer release y el primer sprint (just in time)



La planificación del portafolio es planificar qué producto armar.
Release tiene que ver con producto, el proyecto obtiene como resultado un release del producto, plan de producto y de release es en el ámbito producto, los otros en el ámbito proyecto

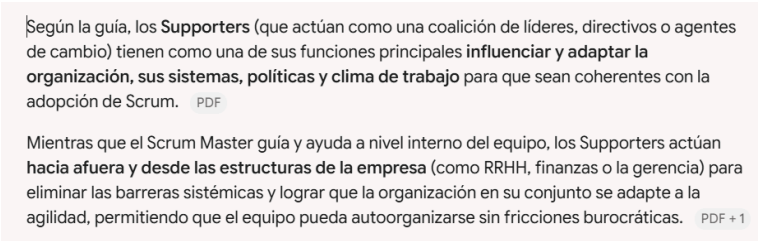
El tablero de scrum solo tiene to do (es el sprint backlog), doing, done. Tablero scrum es el que se usa para el seguimiento y visualización





En tradicional no estimamos directamente esfuerzo, sino que estimamos primero tamaño/alcance (casos de uso, cantidad de requerimientos, clases, la recomendación es tratar de estimar en algo que tengas pronto) que es del producto, después esfuerzo en horas,
En tradicional la estimación tiene foco en la precisión, para este día a esta hora. Si hay un cambio debo re estimar todo
En ágil la estimación apunta a la certeza (lo voy a tener para mayo), estimamos producto
En ágil estimamos producto y en tradicional estimamos producto en el tamaño, en esfuerzo/tiempo y costo estimamos proyecto

En el agilismo como los requerimientos son variables dejo fijo cuanto me cuesta el equipo y cuanto dura el sprint
Supporter se encarga de permitir que un equipo ágil se integre a la organización, si la org no es ágil es más trabajo el que tiene
REVISAR ACTIVIDADES DE LA GESTIÓN DE CONFIGURACIÓN



El Scrum Master se enfoca hacia adentro y hacia el propio equipo, guiándolos en la práctica de Scrum y ayudándolos a mejorar su efectividad.
El Supporter se enfoca hacia afuera y hacia la estructura de la empresa, modificando el entorno, los sistemas y la cultura para que la organización sea compatible con la agilidad y permita que el equipo prospere

---

PARCIAL 2:
Clase testing ágil:
Testing/ prueba: su propósito es encontrar defectos cuya presencia se asume en el software. Una vez que terminó de hacerse el testing, no puede garantizar calidad ni que el producto no tiene más defectos, solo hace evidentes los defectos que pudieron encontrarse. El gran desafío es encontrar los defectos.
NO DECIR GARANTIZAR CALIDAD, **con el testing no aseguramos calidad,** aseguramiento de calidad y testing no es lo mismo. El testing no puede garantizar que el producto no tiene defectos. NO puede certificar calidad.

Calidad: es algo subjetivo, depende de la persona que considera o no como calidad. La adecuación de un producto o servicio a las necesidades o expectativas de quien lo pidió. Nivel del cumplimiento del producto o servicio a las necesidades y expectativas (requerimientos).
Diferencia entre necesidad y expectativas: las expectativas no son manifiestas, son cosas que él quiere pero nunca te lo dijo. Necesidades más cerca de los requerimientos funcionales y expectativas más cerca de los no funcionales.

REVISAR BIEN LOS TÍTULOS CUANDO ESTUDIE.
Haces testing cuando ya tenes una versión del producto, mientras que aseguramiento de calidad es un conjunto de acciones para tratar de que el sw tenga calidad (desarrollo exitoso), puede llevar a un testing no exitoso (porque no encuentra defectos).

En un contexto de PUD: tenemos requerimientos, análisis, diseño, implementación, prueba,
Validación: si el producto es correcto, si estoy haciendo lo que el cliente pide.
Verificación: si el producto funciona correctamente.

Error defecto no son lo mismo, el error es aquel que se detecta cuando queda en el contexto de la misma fase donde se cometió (ejemplo un error de requerimiento que se detecta y corrige en la etapa de requerimiento). Un defecto es aquel que se propagó entre las fases (ejemplo un error de requerimiento que se encontró en diseño). El testing busca encontrar defectos en el código (puedo alguna veces hacer testing de manuales contra el código), el testing necesita código. Decimos que encontramos defectos porque se introdujeron antes del testing y lo mejor es encontrar defectos de

implementación porque son los más fáciles de resolver y menos perjudiciales. Es por esto que se dice que el testing llega tarde, por eso también se puede decir que el testing es reactivo y qa es proactivo.

Para que la disciplina de aseguramiento de calidad funcione, el testing está como una actividad de control de calidad, la disciplina de administración de configuración nos da las necesidades básicas para llevar a cabo el aseguramiento de calidad.
El testing es la actividad más costosa, la gente por eso no hace testing o no todo lo que debería.

Siempre es más barato hacer aseguramiento de calidad para hacer detección temprana de defectos y corregirlos.
Estrategias de caja negra (dinámica): para hacer testing deben ejecutar el código, las de caja blanca son estáticas porque solo miran el código.

Falla no es lo mismo que error o defecto, falla es un mal funcionamiento del sistema que ocurre como consecuencia de un error o defecto. No todos los defectos provocan fallas, por ejemplo defectos de diseño de interfaz, se usaron colores que no corresponden, o nombres que no corresponden para un botón, errores de ortografía.

No todos los defectos dan la misma consecuencia, se los clasifica para organizar su corrección. Aparece el concepto de severidad (cuan grande es el defecto o cuanto me afecta a mi en el sw ese defecto, el cliente solo puede determinar el nivel bloqueante, la severidad la asigna el equipo de testing):

- Bloqueante: no se puede usar el sistema, no anda el sistema.
- Crítico: no puedo cumplir mi objetivo con esta funcionalidad, pero si anda el sistema. No tengo otra manera de hacerlo
- Mayor: no puedo cumplir la funcionalidad pero tengo otra manera de hacerlo. Ejemplo conocer los últimos movimientos, esa funcionalidad no anda pero los puedo consultar por otro lado.
- Menor: no impide cumplir la funcionalidad pero no exactamente en los términos esperados, te da una advertencia pero también te deja continuar o por ejemplo filtra pero muestra con otro orden.
- Cosmético: errores de ortografía, o visuales.

Prioridad tiene que ver con la urgencia en el negocio, lo asigna el cliente cuando lo detecta (lo ideal sería que testing no deje defectos que los usuarios encuentren).

- Urgente: no puedo cobrar en el supermercado, no puedo esperar nada.
- Baja: puede hasta esperar al próximo release para corregirlo

Puede haber un error grave que puede esperar para corregirse.

Los defectos siempre hay que corregirlos pero los corregimos de acuerdo a su prioridad.

Niveles de prueba: tienen un orden de ejecución

- Primer nivel / pruebas unitarias: las hace el desarrollador, son la excepción a la regla que dice que nadie debería probar su propio código, el desarrollador prueba que va a subir al repo para saber si al menos compila. Se hacen en el workflow de implementación.
- Pruebas de integración/ de interfaces: si componentes aislados que funcionan bien, en conjunto funcionan bien también. El resultado es un conjunto de componentes que están integrados

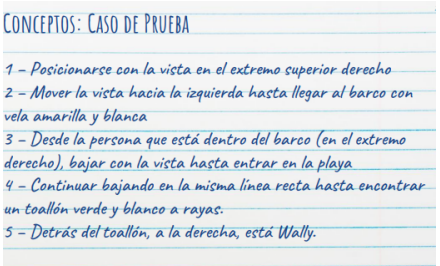
(build). La teoría dice que debería hacerse en el workflow de prueba, pero en la práctica se hace en el ámbito de implementación (ella lo asocia a la implementación). Cuando trabajas con la integración continua automatizar el proceso de prueba de integración porque cada desarrollador sube al repo su parte y se prueba automáticamente.

- Prueba de sistema / de versión: van en el workflow de prueba, la hace alguien distinto a quien construyó, se busca la mayor independencia posible entre el equipo de desarrollo y de testing, es deseable que hasta el jefe sea distinto para que no haya presiones.
- Pruebas de aceptación de usuario: las hace el usuario, de estas pruebas depende que nos acepten el producto. Las hacemos en el despliegue. En las user solo ponemos una descripción de estas pruebas, pero no son como tal las pruebas.

Cada componente que se prueba es una prueba unitaria.
En pruebas de sistema y aceptación el foco es el mismo, se difiere en quien lo prueba. Se prueban porciones verticales.

Ambientes/ entorno: configuración del espacio del trabajo con el repositorio con todas las herramientas que vamos a utilizar en cada momento (ejemplo. Esto se planifica en el plan de gestión de configuración. Una cosa es el ambiente de desarrollo y otra el de testing. No pueden trabajar sobre el mismo entorno los desarrolladores y testing.
Separamos testing de desarrollo porque tenemos que resolver el problema de “en mi maquina andaba” y tienen que trabajar en un espacio donde los desarrolladores no influyan. Y no se podría corregir cosas que el tester está revisando al mismo tiempo, por eso también se separa.

El ambiente de pre-producción debería tener las mismas características y hardware que en producción. Las pruebas de aceptación se hacen en pre-producción, es menos riesgoso. Necesito que sea el mismo hardware para poder probar la performance, estrés. Si la empresa tiene que tener o no todos los ambientes, depende de qué empresa es (si es una consultora que hace software para clientes no tengo producción y preproducción, el banco por ejemplo hace su propio sw y tiene su propio ambiente de producción). Por eso normalmente se hacen solo pruebas funcionales.



Este es un ejemplo de caso de prueba, **el entregable/artefacto más importante del testing es el caso de prueba.**
El caso de prueba nos permite hacer un testing sistemático y metodológico, permite encontrar defectos porque puede reproducirse. Un defecto que no se puede reproducir no es un defecto. Construyo esta secuencia de pasos para que los de testing puedan reproducirlos. Son creadas por los testers, la idea es que alguien los diseñe y otros alguien los ejecute. No hay garantía de que se va a encontrar un defecto.
El testing es muy detallado, se pone por ejemplo la fecha exacta que quiero ingresar, el nombre del usuario específico.

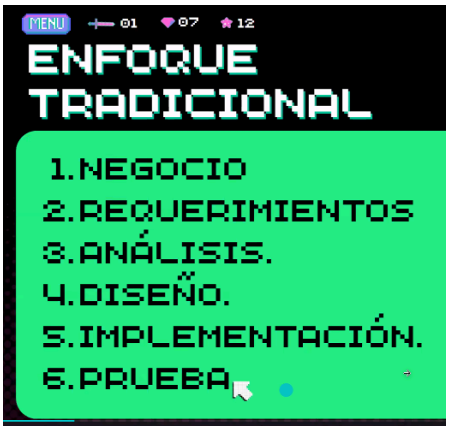
La problemática más importante del testing es que es caro y que las pruebas exhaustivas no se pueden hacer (probar todas las combinaciones). Debemos reducir la cantidad de casos de pruebas sin afectar la posibilidad de encontrar defectos, nos deben ayudar a un testing exitoso sin reducir la calidad del testing.
SOLO VAMOS A HACER PRUEBAS DE CAJA NEGRA.
Si no tengo los requerimientos escritos no puedo hacer validación y verificación. Si uso los casos de prueba solo voy a poder hacer validación.

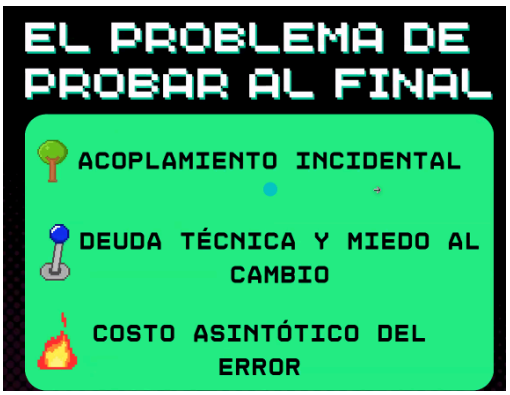
En la planificación de pruebas el artefacto que sale es el plan de pruebas
En diseño de CP: el artefacto es el caso de prueba (si son automatizados los programas), y los datos de prueba (las bases de datos que se usarán para probar)
En estas dos etapas está el 50% del testing y pueden hacerse antes de tener el código, solo teniendo los requerimientos.
Después en la ejecución entra la versión de sw y los casos de prueba y se prueba. La salida normalmente son los defectos pero como no me voy a poner a documentar ahí, se pone como actividad separada.
Después reportes y defectos tiene como artefacto los reportes de defectos.
En tradicional se hace cuando se planifica el proyecto, en el agilismo algunos usan un sprint 0 para armar las pruebas, configurar el entorno (no va a tener producto).

Ciclo de prueba: desde que empiezo a hacer la ejecución y hago todas las pruebas. Termina con el reporte hecho, ese reporte después vuelve a desarrollo para que sean corregidos y desarrollo después manda otra versión ya corregida, testing vuelve a ejecutar las pruebas. El primer ciclo que se ejecuta se llama ciclo 0, en un escenario ideal debería tener 2 ciclos (encuentro defectos, los corrijo y vuelvo a probar y pasa), no es uno solo porque no es compatible con la definición de testing (encontrar defectos).

Pruebas de regresión: cuando a partir del ciclo 1 prueba todos los mismos casos de prueba del ciclo 0 para ver que se resolvieron los defectos y no surgieron nuevos. Si solo pruebo los defectos que ya encontré, serían pruebas sin regresión y no sirve, la estadística dice que por cada defecto corregido surgen 3 nuevos. En la industria se usa más sin regresión porque es más barato, la automatización intenta corregir esto.

CLASE PRÁCTICA TDD:

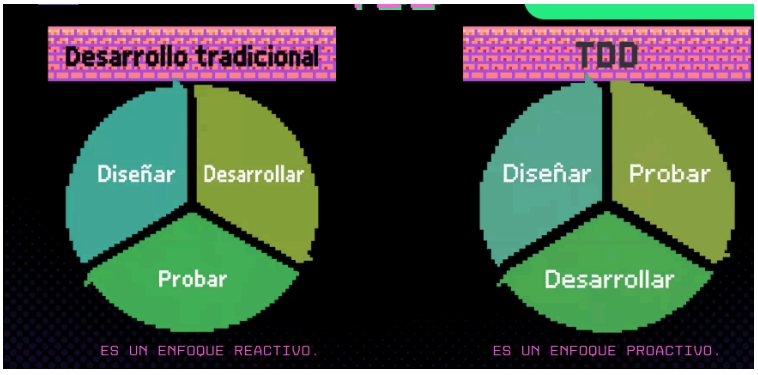




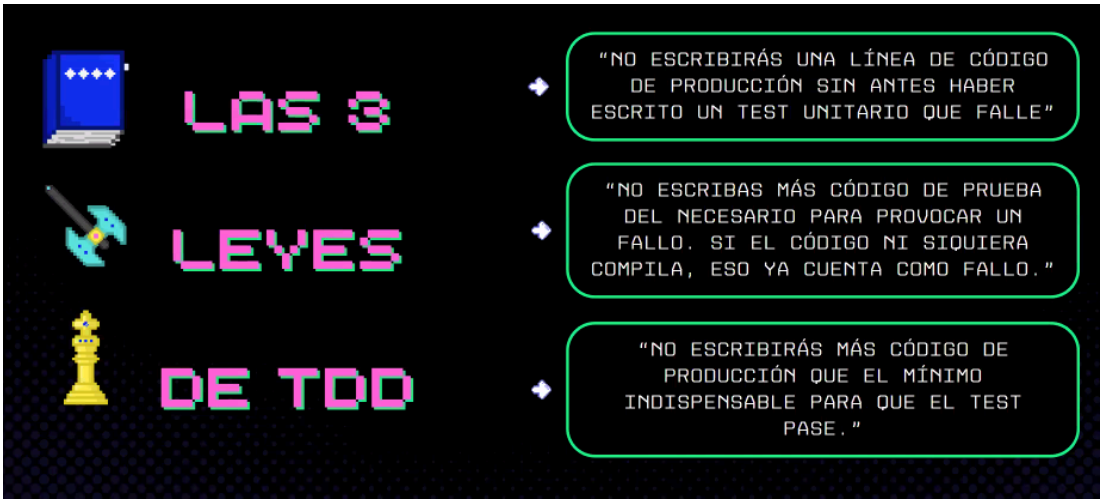
En el enfoque tradicional
Acoplamiento incidental: los desarrolladores hacen el código esperando que funcione, a la hora de diseñarlo no se divide el problema en partes pequeñas, no se tienen en cuenta las partes aisladas, dificultando el testeo. Hay que generar el entorno de prueba, configurarlo, se pierde mucho tiempo.
Problema de deuda técnica y miedo al cambio: a la hora de salir a producción siempre surgen problemas y no podían implementarse fácilmente los cambios y era frágil (al mínimo cambio implicaba retrabajo). Configurar el entorno es muy costoso.
Costo asintótico del error: el costo de arreglar un bug, a medida que pasa el tiempo este va incrementando (si paso por ejemplo 3 meses en producción, los costos incrementan exponencialmente)

Todo esto dio pie al TDD:
TDD no es una práctica de testing, es una práctica de desarrollo que usa testing para desarrollar. Es una actividad de especificación y diseño que guía el desarrollo por medio de micro ciclos de retroalimentación basados en pruebas unitarias automatizadas. Por medio de pruebas podemos definir qué debe hacer el sw y luego definir el código, el cómo. Definimos el éxito para luego ver como llegar a ese éxito.



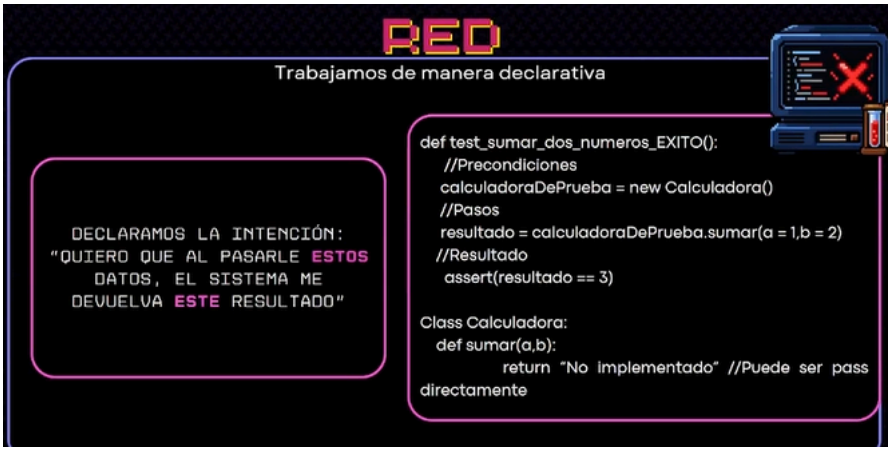


En tradicional desarrollo y después pruebo. El testing era una actividad de verificación una vez que el sw ya estaba hecho.
En TDD primero defino las pruebas que voy a querer desarrollar, toma un enfoque proactivo para adelantarnos a los problemas, más que resolverlos en tiempo de ejecución.



El código resultante en producción debe tener una gran cobertura de pruebas, la idea es que los problemas no surjan en producción.
La 2da ley nos dice hasta dónde escribir una prueba, no deben ser eternas para recibir rápido la retroalimentación, cuando logras un fallo (sería que el código no compile) pasas a la siguiente fase. Los ciclos deben ser lo más corto posibles.
La tercera ley es un enfoque minimalista, la idea es que nos enfoquemos en el ciclo para refactorizar.

En la etapa red escribimos una prueba que falle, quiero pasar ciertos datos y que me de ciertos resultados. Definir qué quiero, después se ve como se hace, es declarativo. Declaro que quiero hacer, nos ayuda a pensar que tenemos que hacer.
En la etapa green resolvemos la falla para que pase la prueba



Ponemos def o test para explicitar un test, le pongo un nombre representativo. Las precondiciones son el armado del entorno que necesito para ejecutar el test (si necesito datos, permisos, si tengo que trabajar con una calculadora tengo que crearla, etc). Los resultados van con assert, declaro el valor que quiero que me devuelva, eso me determina si un test pasa o no.
Esta prueba fallará porque esperaba un 3, y recibí un mensaje. Nos destraba el camino para conseguir el objetivo.

En la etapa green vamos a escribir el cómo, nos enfocamos a escribir lo mínimo para que pase la prueba. Hay patrones que nos definen cuándo pasar a otra etapa, hay un patrón fake it que me permite pasar rápido a la otra etapa. Lo fuerzo a que devuelva 3, la idea de esto es que un problema complejo lo dividamos en partes pequeñas para que no se escape nada.
Después viene la etapa de re factory donde acá si usamos principios/ patrones, modificamos la estructura interna sin alterar el comportamiento externo, pero no se debe modificar a nivel lógico sino a nivel de buenas prácticas para que no falle el test con los cambios.

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    // //Pasos
    resultado = calculadoraDePrueba.sumar(a = 1,b = 2)
    //Resultados
    assert(resultado == 3)

Class Calculadora:
   def sumar(a: Number,b: Number) -> Number:
       return 3
```

la salida no cambia, después paso de ciclo (paso por las 3 etapas). Le puedo cambiar los parámetros de entradas por ende el código anterior fallará, esto me indica que tengo que generalizar el problema que en primera instancia generalice y cambio el código de la suma. En cada ciclo voy a ir generalizando más.



Los patrones nos ayudan a que el ciclo funcione y no detenernos en el desarrollo
Patrones de red, nos ayuda a definir qué hacer:

- Starter test: nos dice que elijamos la prueba más fácil para que el flujo comience, nos ayuda a destrabar el comienzo.
- One step test: segui con el segundo más simple, paso a paso para cumplir las metas.
- Explanation test: se usa en una etapa madura del ciclo, cuando trabajamos en equipo y surgen debates sobre el comportamiento del sistema, definamos el debate como un test para clarificar. Es más de un comportamiento de negocio por ejemplo, debate de como hacer la suma.
- Learning test: nos dice que si quiero por ejemplo conectarme con un sistema externo, representemos el aprendizaje con un test. Armemos un test para entender cómo funciona la API, cómo reacciona nuestro sistema, para luego agregarlo formalmente al sistema. Se enfoca en lo que es cuando quiero integrar sistemas o herramientas de terceros, que mejor que aprender algo probandolo.
- Another test: para la gente que se distrae mucho, y cuando te alejas del camino (estas haciendo a y pensas en como hacer b), se para en ese momento lo anote en la lista y siga con el test que estaba trabajando.

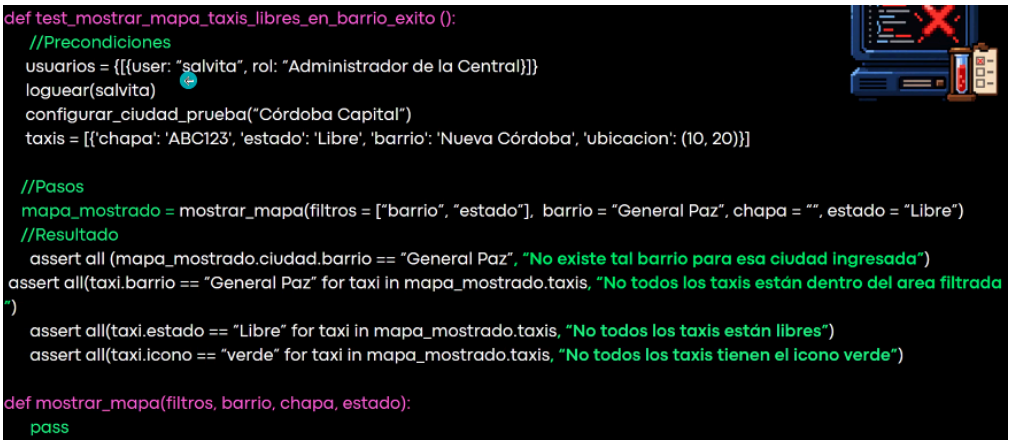
- Regression test: cuando ocurre un problema, voy a definir un test que replique los pasos que ocasionaron ese problema para que no vuelva a pasar en un futuro. Cuando ocurre, se frena y se toma el paso a paso.

Patrones green: formas en la que puedo hacer que una prueba pase:

- Fake it: voy a hardcodear para que la prueba de lo que tiene que dar.
- Triangulación: armo varios casos de la falla para que un problema particular se puede llevar a genérico para tener la certeza que está en el código/logica el problema.
- Obvious implementation: si ya estás seguro de lo que decis, escribí la solución tal cual es. Funciona en problemas sencillos o casi triviales.
- One to many: primero nos enfocamos en que funcione con una unidad y después con una colección, para que pase de sumar dos números a un conjunto de números.

Diseño de test:

- Test list: antes de desarrollar defino qué pruebas debo realizar.
- Isolated test: los patrones deben ser independientes del resto, es una característica.
- Test first: no vas a escribir código hasta no tener las pruebas, de la mano de la primera ley.
- Assert test: cuando me cuesta escribir un test por la complejidad que tiene, propone que arranquemos por lo que queremos probar. Ir desde el final al principio
- Test data: la info que vamos a usar tiene que ser la más sencilla.
- Evident data: la relación entre entradas y salidas debe ser la más obvia, si necesito calcular el iva le doy una entrada de 100 porque es el cálculo más rápido.



```
def test_mostrar_mapa_taxis_libres_en_barrio_exito ():
    //Precondiciones
    usuarios = {[{user: "salvita", rol: "Administrador de la Central}]}
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'Nueva Córdoba', 'ubicacion': (10, 20)}]

   //Pasos
   mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"],  barrio = "General Paz", chapa = "", estado = "Libre")
   //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area filtrada
")
    assert all(taxi.estado == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    pass
```

GREEN

```
def test_mostrar_mapa_taxis_libres_en_barrio_exito ():
    //Precondiciones
    usuarios = {[{user: "salvita", rol: "Administrador de la Central}]}
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
   //cargo los datos al sistema.
    taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'General Paz', 'ubicacion': (10, 20)}]

   //Pasos
   mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"],  barrio = "General Paz", chapa = "", estado = "Libre")
   //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area
filtrada ")
    assert all(taxi.estado.nombre == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.estado.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    return new Mapa([taxi={chapa = "ABC123", estado = {nombre = "Libre", icono = "verde"}, barrio = "General Paz"})
```

REFACTOR

```
def mostrar_mapa(filtros: String[], barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
    estado_libre = new Estado(
        nombre = "Libre",
        icono = "verde"
    )

    barrio_general_paz = new Barrio(
        nombre = "General Paz"
    )

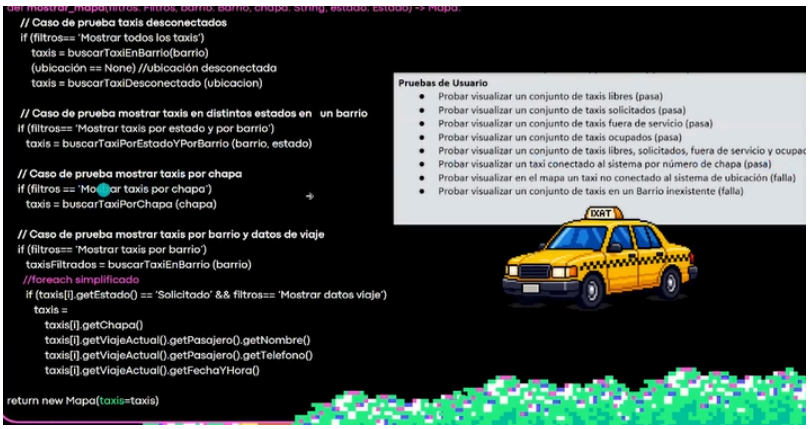
    taxi = new Taxi(
        chapa = "ABC123",
        estado = estado_libre,
        barrio = barrio_general_paz
    )

    taxis = [taxi]

    return new Mapa(taxis = taxis)
```

1. Volver a inciar el ciclo.
2. Considerar el resto de casos de pruebas, añadiendo las alternativas planteadas en la user.
3. Refactorizar.

Hacer TDD sin refactorizar es simplemente escribir código malo con pruebas.



Se puede probar primero campo por campo y después la funcionalidad del sistema.